

```
In [1]: #modules:
import numpy as np
import astropy.units as u
import astropy.constants as const
import matplotlib.pyplot as plt
from scipy import optimize
import sympy
sympy.init_printing()
from scipy.optimize import curve_fit
import emcee
import corner
```

```
In [2]: #The following model was obtained from Mandel et al., 2002
#This model returns the relative flux from the normalized separation z, relative
#This model is only used for plotting the relative flux as a function of z to 1

def MandelModel(p,z, carr):
    #setting up variables used in the model
    z = abs(z)
    r = sympy.symbols("r")
    Omega = np.sum(carr/(4.0+np.arange(len(carr))))
    a = (z-p)**2
    mu = (1-(r**2))**(1/2)

    #setting up integral:
    Isum = np.sum((carr[1:5]*(1-(mu**((np.arange(len(carr)-1)+1)/2))))))
    Ir = (1 - Isum)*2*r

    #model conditions:
    b = 1+p
    c = 1-p

    #condition 1:
    if z > c and z < b:
        #integrating with respect to r, with limits z-p and 1
        Iz = sympy.integrate(Ir, (r, z-p, 1))
        Iz = ((1-a)**(-1))*Iz
        F4 = (Iz)/(4*np.pi*Omega)
        F1 = ((p**2)-((z-1)**2))**(1/2)
        F2 = (z-1)
        F3 = (p**2)*np.arccos((z-1)/p)
        F = 1 - F4*(F3 - (F2*F1))
        return F
    #condition 2:
    elif z < c:
        #integrating with respect to r, with limits z-p and z+p
        Iz = sympy.integrate(Ir, (r, z-p, z+p))
        Iz = ((4*z*p)**(-1))*Iz
        F4 = (Iz)/(4*Omega)
        F = 1 - ((p**2)*F4)
        return F
    #condition 3:
    else:
        Iz = 0
        F = 1 - ((p**2)*Iz)/(4*Omega)
        return F
```

```
In [ ]: #Plotting relative flux as a function of normalized separation to understand the
        #matrix of nonlinear limb darkening coefficients: cmat[0]: c1=c2=c3=c4=0, cmat[1:]
        cmat = np.eye(5)

        #array of 100 z values between -1.2 and 1.2
        z = np.linspace(-1.2, 1.2, 100)

        #plotting
        fig, ax = plt.subplots(1, 1, figsize=(15, 5))
        ax.set_xlim([-1.2, 1.2])
        ax.set_ylim([0.980, 1.001])
        ax.set_xlabel(r'$z$')
        ax.set_ylabel(r'$F$')
        ax.set_title('Relative Flux as a Function of z')
        for b in range(len(cmat)):
            carr = cmat[b]
            G = []
            for a in z:
                f = MandelModel(0.1, a, carr)
                G.append(f)
            ax.plot(z, G, label = b)
        fig.legend(fontsize=12)
```

```
In [3]: #Plotting raw data: relative flux vs time, this function also returns relative
def read_transit(file='tres-2b_transit.txt', plot='y', display='n'):
    """
    Function read_transit reads a light curve file downloaded from the NASA Exoplanet
    IPAC Table Format and returns light curve data in a 2D array with three columns:
    1. Heliocentric Julian Day
    2. Relative Flux
    3. Relative Flux Uncertainty

    Arguments to read_transit include:
    file - filename of the light curve in IPAC Table Format
    display - if 'y' the data in the IPAC Table is printed (not just the 3 columns)
    plot - if 'y' the light curve is plotted, otherwise no plot

    The IPAC Table Format has keyword and comment rows that begin with ";"
    followed by header rows beginning with "|"
    followed by data in columns separated by whitespace.
    Different files may contain different numbers of columns in different order.
    For example, some files contain relative flux and some contain magnitude.
    Unfortunately, this means each light curve file must be examined and this function
    must be modified to return the appropriate data.

    The light curves used in Holman et al. 2007 which we are interested in have:
    1. Heliocentric Julian Day
    2. Relative Flux
    3. Relative Flux Uncertainty
    4. Accepted

    The light curve IPAC Tables for Holman all have the data starting after row 137.
    Other light curve files will likely have the data starting on a different row.
    This function will likely need to change.

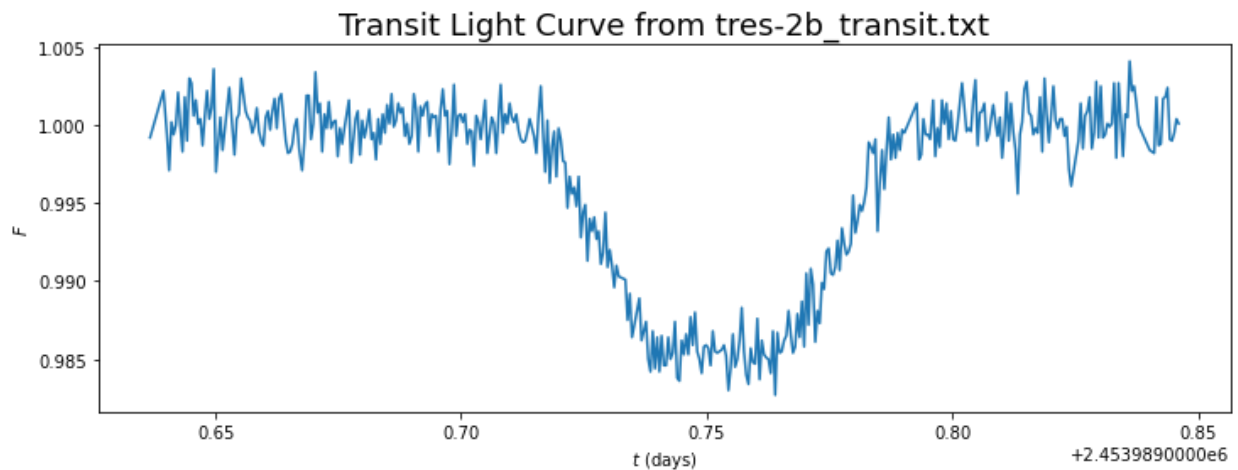
    """
    try:
        light_curve = np.loadtxt(file, skiprows=137) # adjust for different files
```

```

except:
    print(f'Unable to read light curve from {file}.')
assert len(light_curve.shape) == 2, (
    f'The Light curve from {file} should be a 2D array, not shape {light_curve.shape}')
assert light_curve.shape[1] == 4, ( # adjust for different files
    f'Light curve has {light_curve.shape[1]} columns. Which columns are Data and Time?')
if display == 'y':
    print(f'Data from {file}:')
    print(light_curve)
if plot == 'y':
    plt.figure()
    plt.figure(figsize=(12,4))
    plt.title(f'Transit Light Curve from {file}', fontsize=18)
    plt.xlabel(r'$t$ (days)')
    plt.ylabel(r'$F$')
    plt.plot(light_curve[:,0],light_curve[:,1]) # adjust for different files
    return light_curve[:, 0:3] # adjust for different files
data = read_transit(file='tres-2b_transit.txt')

```

<Figure size 432x288 with 0 Axes>



```

In [4]: #Light curve model provided by Professor Burkhart
#This model utilises numerical integration instead of symbolic integration and
def calcF(fitP, inP):
    ar = fitP[0]
    p = fitP[1]
    i = fitP[2]
    c1 = fitP[3]
    c2 = fitP[4]
    t = inP[0]
    w = inP[1]
    c3 = inP[2]
    c4 = inP[3]
    c0 = 1-c1-c2-c3-c4
    carr = [c0, c1, c2, c3, c4]
    z = (ar)*(np.sin(w*t)**2+(np.cos(i)*np.cos(w*t))**2)**(1/2)
    zz = np.abs(z)
    Omega = np.sum(carr/(4.0+np.arange(len(carr))))
    if zz<1.0-p:
        fac = p**2
        narr = np.arange(len(carr))
        rlo = zz-p #lower limit
        Ilo = rlo**2 + 4.0*(1.0-rlo**2)**(1.0+0.25*narr)/(4.0+narr) #integral a
        rhi = zz+p #upper limit
        Ihi = rhi**2 + 4.0*(1.0-rhi**2)**(1.0+0.25*narr)/(4.0+narr) #integral a

```

```

Istar = rhi**2 - rlo**2 - np.sum(carr*(Ihi-Ilo)) #limit version of inte
Istar /= 4.0*zz*p
elif zz<1.0+p:
    fac = p**2*np.arccos((zz-1.0)/p) - (zz-1.0)*np.sqrt(p**2-(zz-1.0)**2)
    fac = fac/np.pi # it seems to me the Mandel & Agol paper missed a fact
    narr = np.arange(len(carr))
    rlo = zz-p
    Ilo = rlo**2 + 4.0*(1.0-rlo**2)**(1.0+0.25*narr)/(4.0+narr)
    rhi = 1.0
    Ihi = rhi**2 + 4.0*(1.0-rhi**2)**(1.0+0.25*narr)/(4.0+narr)
    Istar = rhi**2 - rlo**2 - np.sum(carr*(Ihi-Ilo))
    Istar /= 1.0-(zz-p)**2
else:
    fac = 0.0
    Istar = 0.0
ans = 1.0 - fac*Istar/(4.0*Omega)
return ans

def calc_lcrv(fitP, inP):
    ans = np.zeros(len(inP))
    for i in range(len(inP)):
        inP1 = inP[i]
        ans[i] = calcF(fitP, inP1)
    return ans

```

```

In [12]: #Chi2 optimization
#the following optimization uses the Model provided by Professor Burkhardt

#Reading experimental data file
light_curve = read_transit(file='tres-2b_transit.txt', plot='y', display='n')
#array of time from centre of transit
t = light_curve[:,0] - 2453989.75286
#orbital frequency
w = 2*np.pi/(2.47063)
#nonlinear limb darkening coefficients
c3 = 0
c4 = 0

#creating inP array
inP = []
for k in range(len(t)):
    inP.append([t[k], w, c3, c4])

#Best guess
inc = np.pi/2
fitGuess = [8, 0.1, inc, 0.5, 0.5]

#Experimental data and uncertainty
lcrvData = light_curve[:,1]
sigma = light_curve[:,2]

#Theoretical data
lcrvModel = calc_lcrv(fitGuess, inP)

#Plotting theoretical data based on best guess
plt.figure(figsize=(9,6))
plt.plot(t,lcrvModel)
plt.legend(fontsize=12)
plt.xlabel(r'$t$ (days)')

```

```

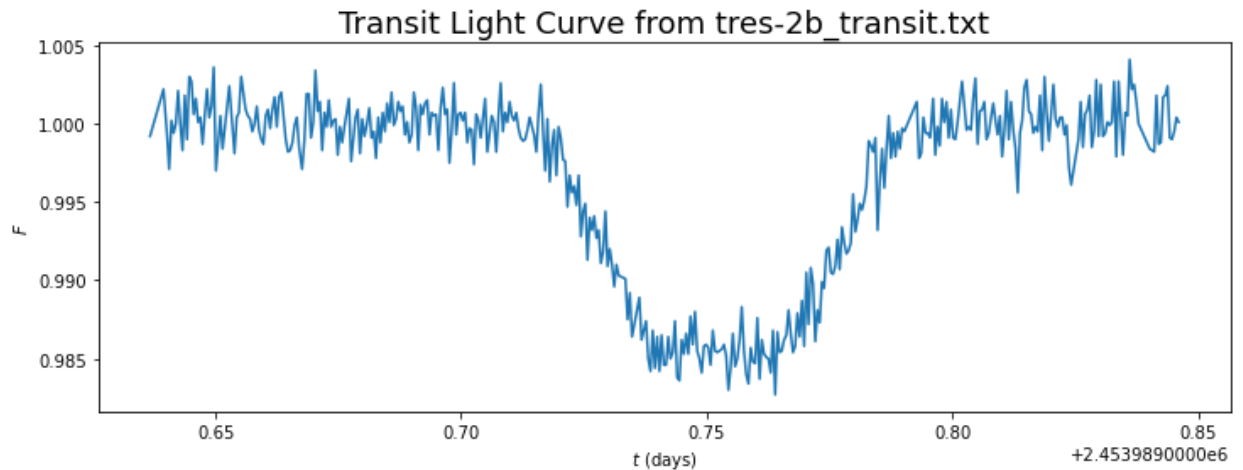
plt.ylabel(r'$F$')
plt.title('Relative Flux Model Based on Best Guess Parameters for Tres-2b')
plt.show()

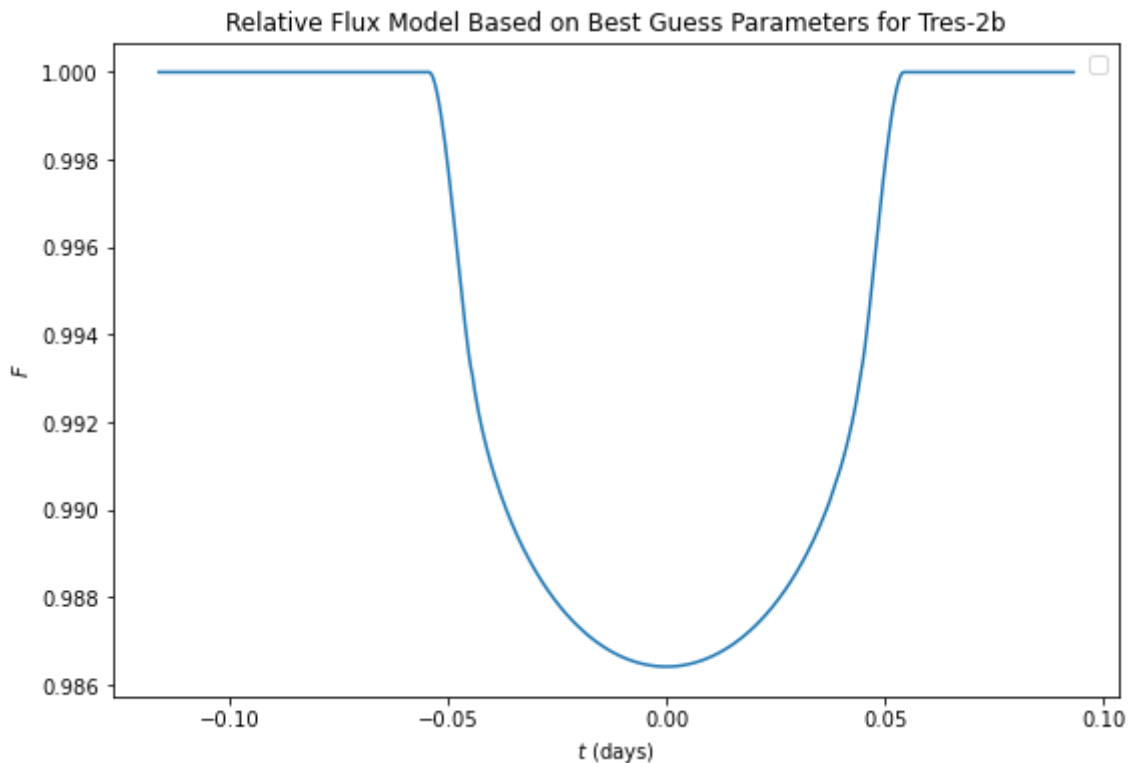
def Chi2Min(fitGuess, inP, lcrvData, sigma):
    chi2 = 0
    lcrvModel = calc_lcrv(fitGuess, inP)
    if (fitGuess[0] < 0) or (fitGuess[1] < 0) or (fitGuess[2] < 0) or (fitGuess[3] < 0):
        return 10**8
    if (fitGuess[3] > 1) or (fitGuess[4] > 1):
        return 10**8
    else:
        for x in range(len(inP)):
            chi2 = chi2 + ((lcrvData[x]-lcrvModel[x])**2/(sigma[x]**2))
        return chi2
pBest = optimize.minimize(Chi2Min, fitGuess, (inP, lcrvData, sigma))
print(pBest)

```

WARNING:matplotlib.legend:No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

<Figure size 432x288 with 0 Axes>





```
/var/folders/lh/wvml1fys14jd28h4cky_5hgm0000gn/T/ipykernel_7107/1408344832.py:
22: RuntimeWarning: invalid value encountered in power
    ilo = rlo**2 + 4.0*(1.0-rlo**2)**(1.0+0.25*narr)/(4.0+narr) #integral at low
er limit
```

```
fun: 481.200744296819
hess_inv: array([[ 9.13772612e-03,  6.21771934e-05,  3.67206584e-05,
  2.73011684e-02, -2.12026683e-02],
 [ 6.21771934e-05,  4.94845489e-07,  3.39930984e-07,
  1.29836932e-04, -1.11208016e-04],
 [ 3.67206584e-05,  3.39930984e-07,  8.14483907e-07,
  1.86002040e-04, -1.43890193e-04],
 [ 2.73011684e-02,  1.29836932e-04,  1.86002040e-04,
  2.37741918e-01, -1.65059078e-01],
 [-2.12026683e-02, -1.11208016e-04, -1.43890193e-04,
  -1.65059078e-01,  1.16106601e-01]])
jac: array([ -40.51276779, -6860.31267548, -833.42346573, -208.551685
33,
  -340.03274918])
message: 'Desired error not necessarily achieved due to precision loss.'
nfev: 526
nit: 7
njev: 86
status: 2
success: False
x: array([11.24190569,  0.10286771,  1.60751602,  0.02033521,  0.98774
15 ])
```

```
In [7]: #The following section implements an emcee algorithm to ascertain the uncertain
ndim = 5 #no. of parameters
nwalk = 10 #no. of walkers
nburn = 1000 #when the burn in run should come in
nmain = 5000 #step size

#calculating relative flux using best fit parameters
lcrvModel = calc_lcrv(pBest.x, inP)
```

```

#plotting experimental data and theoretical values to compare
plt.figure(figsize=(9,6))
plt.plot(t,lcrvModel)
plt.plot(t,lcrvData)
plt.legend(fontsize=12)
plt.xlabel(r'$t$ (days)')
plt.ylabel(r'$F$')
plt.title('Experimental Data and Model Data')
plt.show()

#defining likelihood function
def lnprob(fitGuess, inP, lcrvData, sigma):
    chi2 = -0.5*Chi2Min(fitGuess, inP, lcrvData, sigma)
    return chi2

# random starting points; we will take the strategy of perturbing some starting
pstart = pBest.x #starting point based on chi2 optimization
#small perturbations
p0 = np.array([ pstart+1.0e-4*np.random.normal(size=ndim) for iwalk in range(nw

# set up the sampler
sampler = emcee.EnsembleSampler(nwalk,ndim,lnprob, args = (inP, lcrvData, sigma

# burn-in run
print('burn-in run')
pos,prob,state = sampler.run_mcmc(p0,nburn)

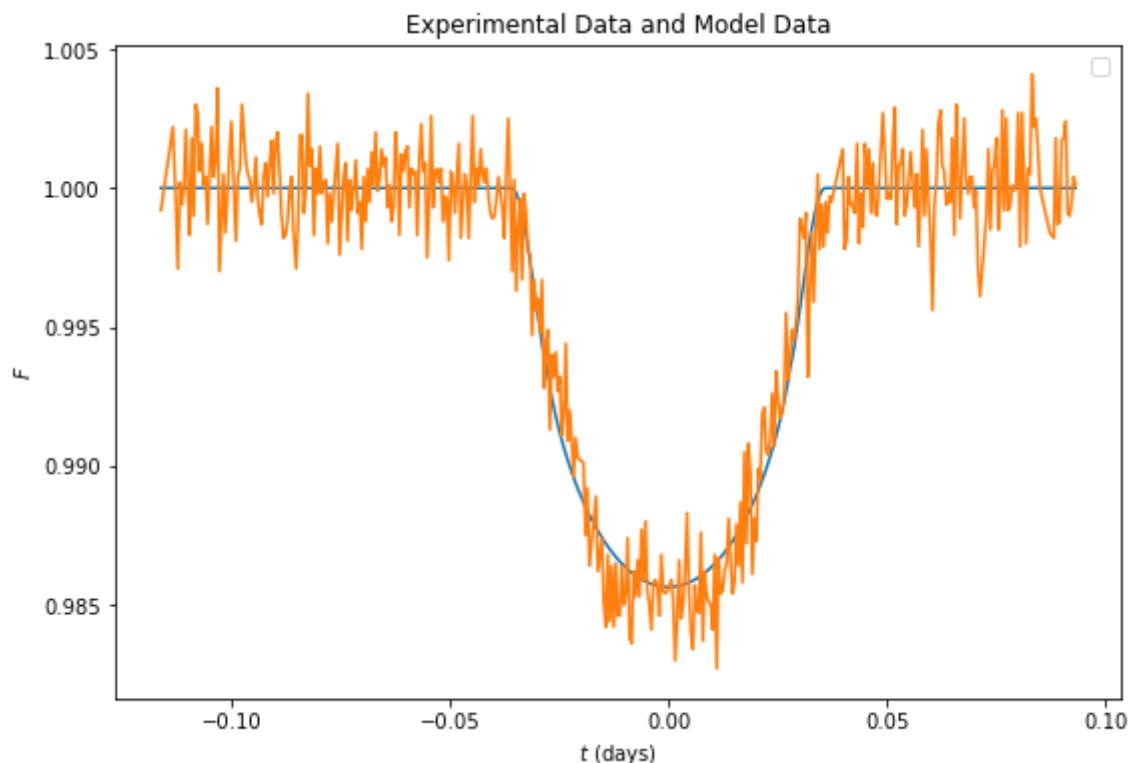
# reset the sampler
sampler.reset()

# main run
print('main run')
res = sampler.run_mcmc(pos,nmain)
samples = sampler.chain.reshape((-1,ndim))

# plot the individual parameters
f,ax = plt.subplots(ndim,1,figsize=(10,10))
for idim in range(ndim):
    for iwalk in range(nwalk):
        ax[idim].plot(sampler.chain[iwalk,:,idim])
        ax[idim].set_xlabel('step')
        ax[idim].set_ylabel('parameter {}'.format(idim+1))
f.show()
f = corner.corner(samples,show_titles=True,labels=('$a_r/r_*$','$p$','$i$','$c_
f.show()

```

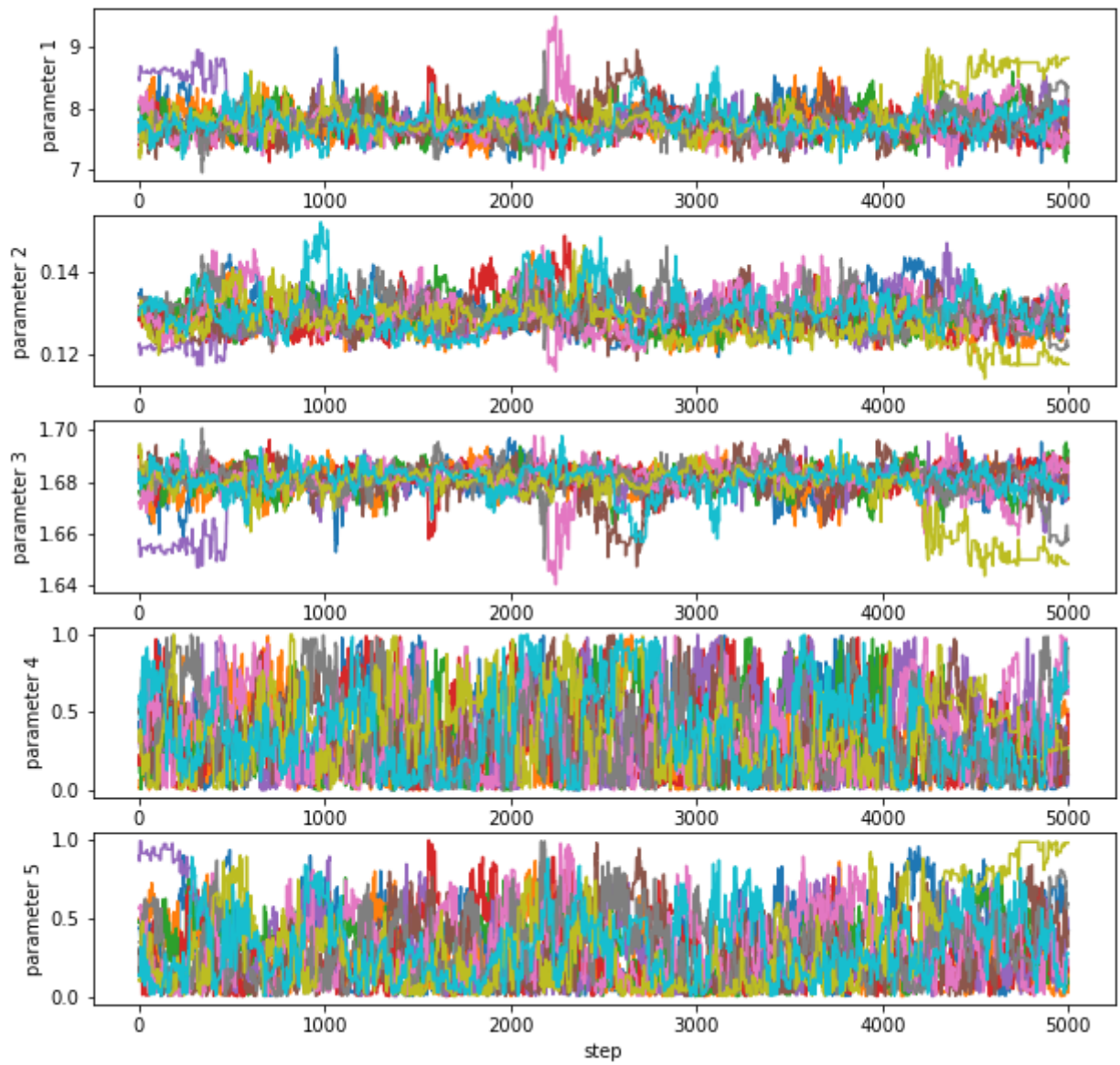
WARNING:matplotlib.legend:No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

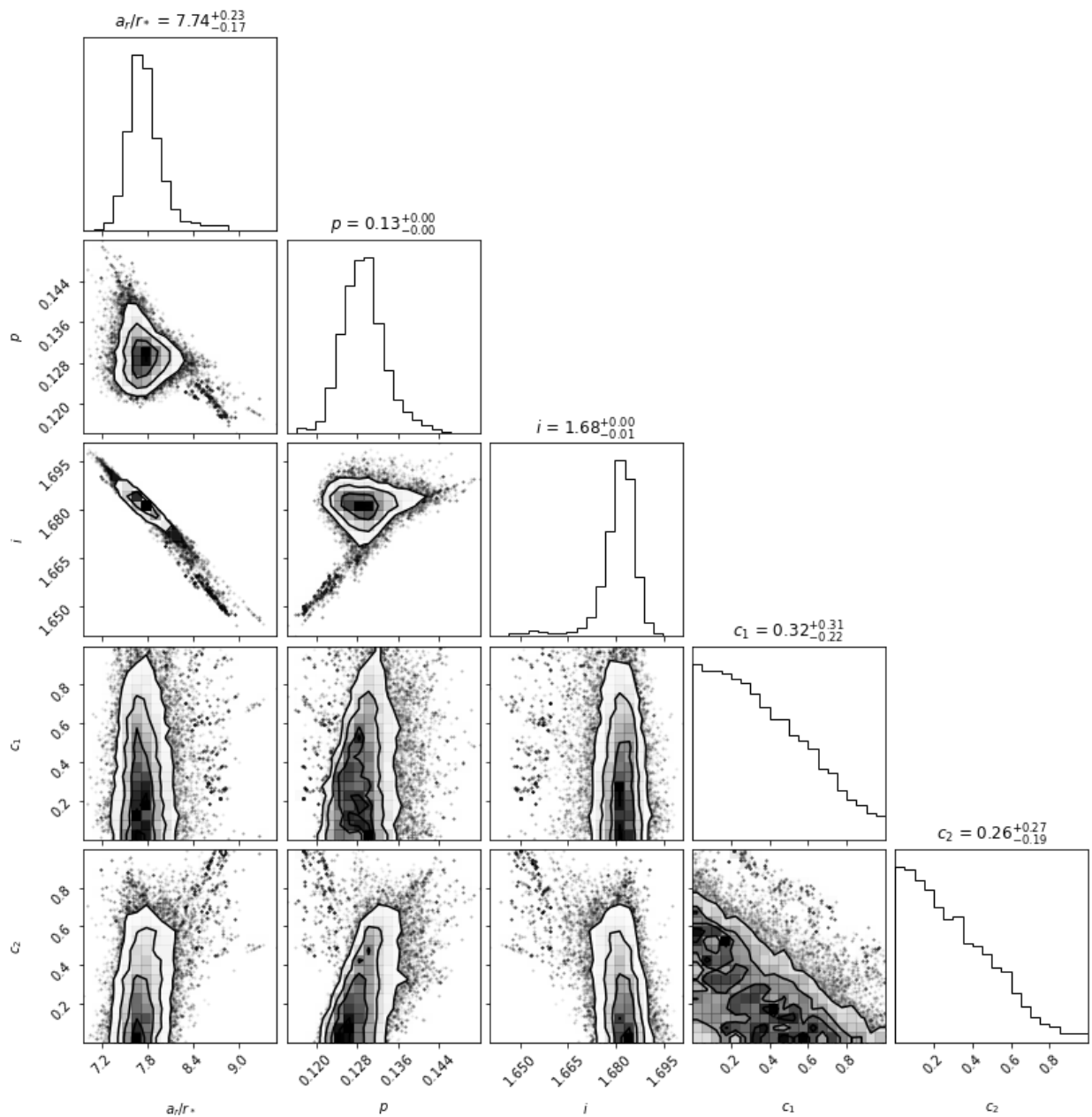


burn-in run

main run

```
/var/folders/1h/wvml1fys14jd28h4cky_5hgm0000gn/T/ipykernel_7107/1939529578.py:
52: UserWarning: Matplotlib is currently using module://matplotlib_inline.back
end_inline, which is a non-GUI backend, so cannot show the figure.
    f.show()
/var/folders/1h/wvml1fys14jd28h4cky_5hgm0000gn/T/ipykernel_7107/1939529578.py:
54: UserWarning: Matplotlib is currently using module://matplotlib_inline.back
end_inline, which is a non-GUI backend, so cannot show the figure.
    f.show()
```



```
In [16]: #The following code details the same process for XO-1b

#Chi2 optimization
#the following optimization uses the Model provided by Professor Burkhardt

#Reading experimental data file
light_curve = read_transit(file='XO-1b_transit.txt', plot='y', display='n')
#array of time from centre of transit
t = light_curve[:,0] - 2453883.80565
#orbital frequency
w = 2*np.pi/(3.941534)
#nonlinear limb darkening coefficients
c3 = 0
c4 = 0

#creating inP array
inP = []
for k in range(len(t)):
    inP.append([t[k], w, c3, c4])
```

```

#Best guess
inc = np.pi/2
fitGuess = [11, 0.135, inc, 0.5, 0.2]

#Experimental data and uncertainty
lcrvData = light_curve[:,1]
sigma = light_curve[:,2]

#Theoretical data
lcrvModel = calc_lcrv(fitGuess, inP)

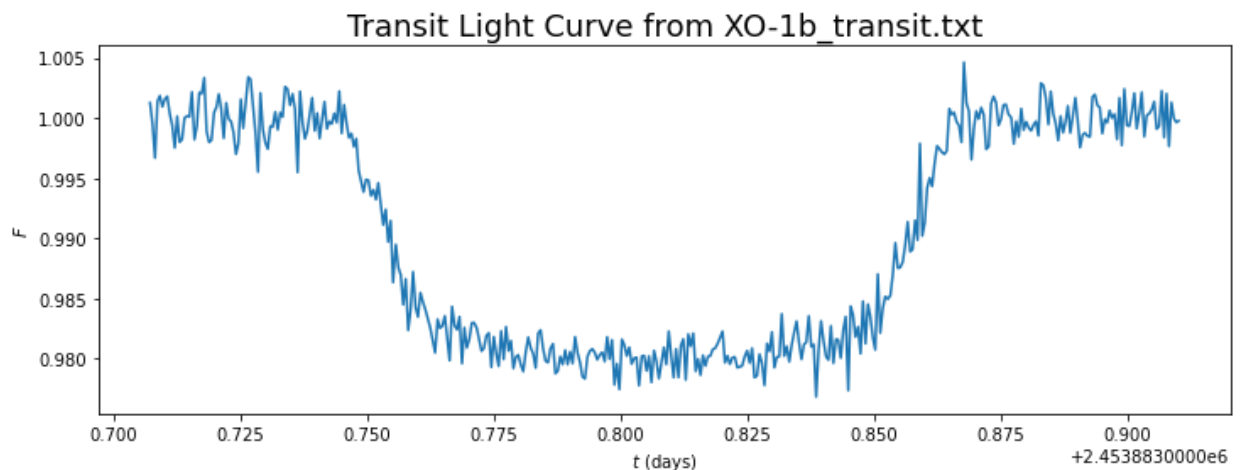
#Plotting theoretical data based on best guess
plt.figure(figsize=(9,6))
plt.plot(t,lcrvModel)
plt.legend(fontsize=12)
plt.xlabel(r'$t$ (days)')
plt.ylabel(r'$F$')
plt.title('Relative Flux Model Based on Best Guess Parameters for XO-2b')
plt.show()

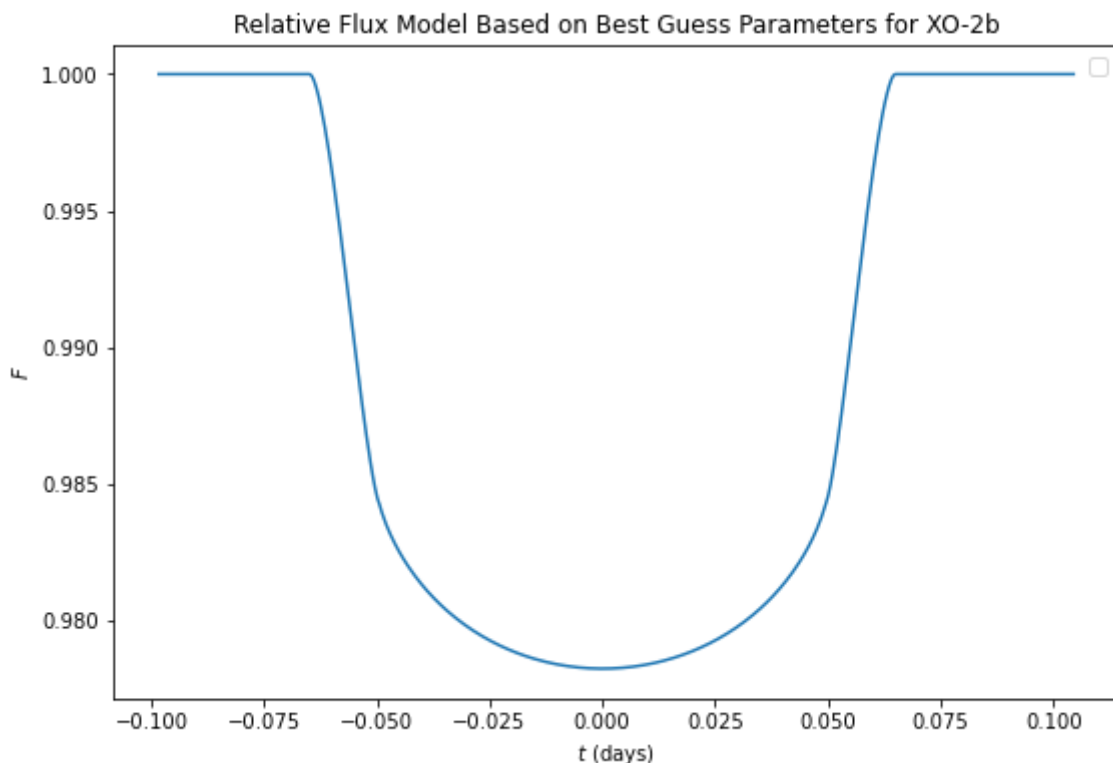
def Chi2Min(fitGuess, inP, lcrvData, sigma):
    chi2 = 0
    lcrvModel = calc_lcrv(fitGuess, inP)
    if (fitGuess[0] < 0) or (fitGuess[1] < 0) or (fitGuess[2] < 0) or (fitGuess[3] > 1) or (fitGuess[4] > 1):
        return 10**8
    else:
        for x in range(len(inP)):
            chi2 = chi2 + ((lcrvData[x]-lcrvModel[x])**2/(sigma[x]**2))
        return chi2
pBest = optimize.minimize(Chi2Min, fitGuess, (inP, lcrvData, sigma))
print(pBest)

```

WARNING:matplotlib.legend:No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

<Figure size 432x288 with 0 Axes>





```
/var/folders/lh/wvmlfys14jd28h4cky_5hgm0000gn/T/ipykernel_7107/1408344832.py:
22: RuntimeWarning: invalid value encountered in power
```

```
    ilo = rlo**2 + 4.0*(1.0-rlo**2)**(1.0+0.25*narr)/(4.0+narr) #integral at low
er limit
```

```
fun: 419.60208636992786
hess_inv: array([[ 4.58613221e-03,  4.48735689e-05, -2.02704486e-05,
                  -3.36644216e-02,  1.88127611e-02],
                 [ 4.48735689e-05,  4.74515790e-07, -1.23503089e-07,
                  -2.91389808e-04,  1.59479097e-04],
                 [-2.02704486e-05, -1.23503089e-07,  2.01106344e-06,
                  1.57451287e-04, -9.19136594e-05],
                 [-3.36644216e-02, -2.91389808e-04,  1.57451287e-04,
                  3.35845071e-01, -1.95304784e-01],
                 [ 1.88127611e-02,  1.59479097e-04, -9.19136594e-05,
                  -1.95304784e-01,  1.14183381e-01]])
jac: array([ 15.48175049, -266.83948517,  151.76360321,  60.8937912 ,
            110.29083252])
message: 'Desired error not necessarily achieved due to precision loss.'
nfev: 420
nit: 7
njev: 69
status: 2
success: False
x: array([11.52469288,  0.13158261,  1.58285692,  0.67810576,  0.06245
358])
```

```
In [17]: #The following section implements an emcee algorithm to ascertain the uncertain
ndim = 5 #no. of parameters
nwalk = 10 #no. of walkers
nburn = 1000 #when the burn in run should come in
nmain = 5000 #step size

#calculating relative flux using best fit parameters
lcrvModel = calc_lcrv(pBest.x, inP)
```

```

#plotting experimental data and theoretical values to compare
plt.figure(figsize=(9,6))
plt.plot(t,lcrvModel)
plt.plot(t,lcrvData)
plt.legend(fontsize=12)
plt.xlabel(r'$t$ (days)')
plt.ylabel(r'$F$')
plt.title('Experimental Data and Model Data')
plt.show()

#defining likelihood function
def lnprob(fitGuess, inP, lcrvData, sigma):
    chi2 = -0.5*Chi2Min(fitGuess, inP, lcrvData, sigma)
    return chi2

# random starting points; we will take the strategy of perturbing some starting
pstart = pBest.x #starting point based on chi2 optimization
#small perturbations
p0 = np.array([ pstart+1.0e-4*np.random.normal(size=ndim) for iwalk in range(nw

# set up the sampler
sampler = emcee.EnsembleSampler(nwalk,ndim,lnprob, args = (inP, lcrvData, sigma

# burn-in run
print('burn-in run')
pos,prob,state = sampler.run_mcmc(p0,nburn)

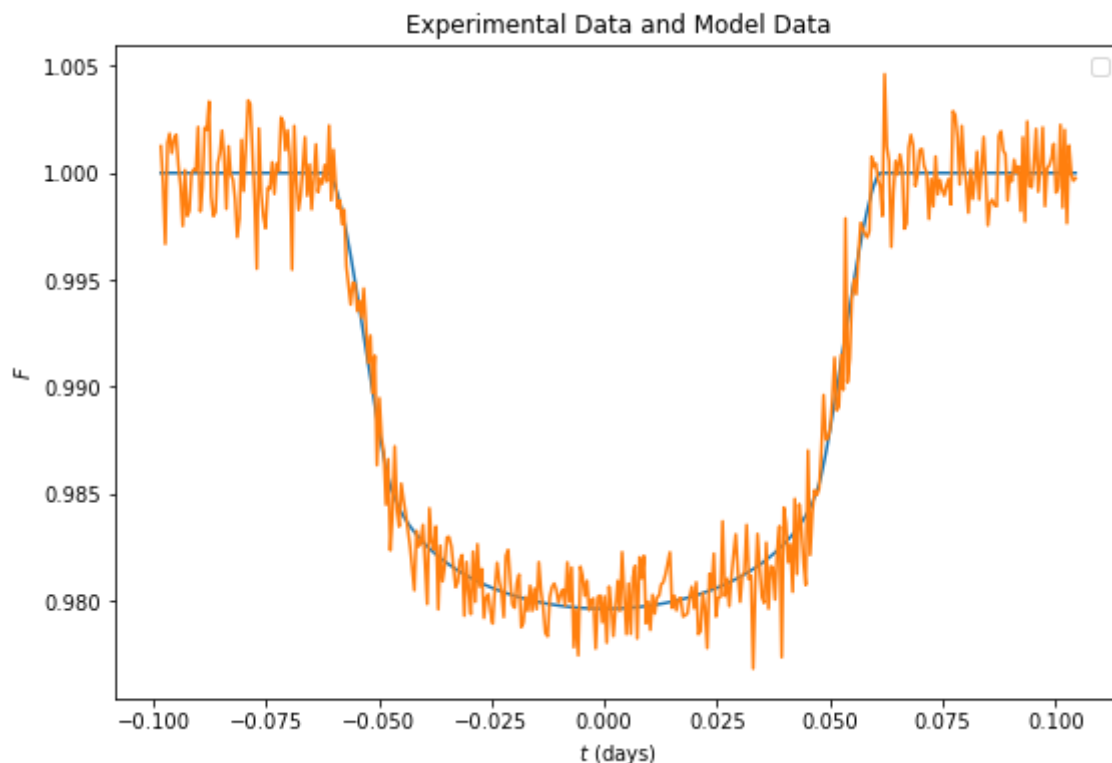
# reset the sampler
sampler.reset()

# main run
print('main run')
res = sampler.run_mcmc(pos,nmain)
samples = sampler.chain.reshape((-1,ndim))

# plot the individual parameters
f,ax = plt.subplots(ndim,1,figsize=(10,10))
for idim in range(ndim):
    for iwalk in range(nwalk):
        ax[idim].plot(sampler.chain[iwalk,:,idim])
        ax[idim].set_xlabel('step')
        ax[idim].set_ylabel('parameter {}'.format(idim+1))
f.show()
f = corner.corner(samples,show_titles=True,labels=('$a_r/r_*$','$p$','$i$','$c_
f.show()

```

WARNING:matplotlib.legend:No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.



burn-in run

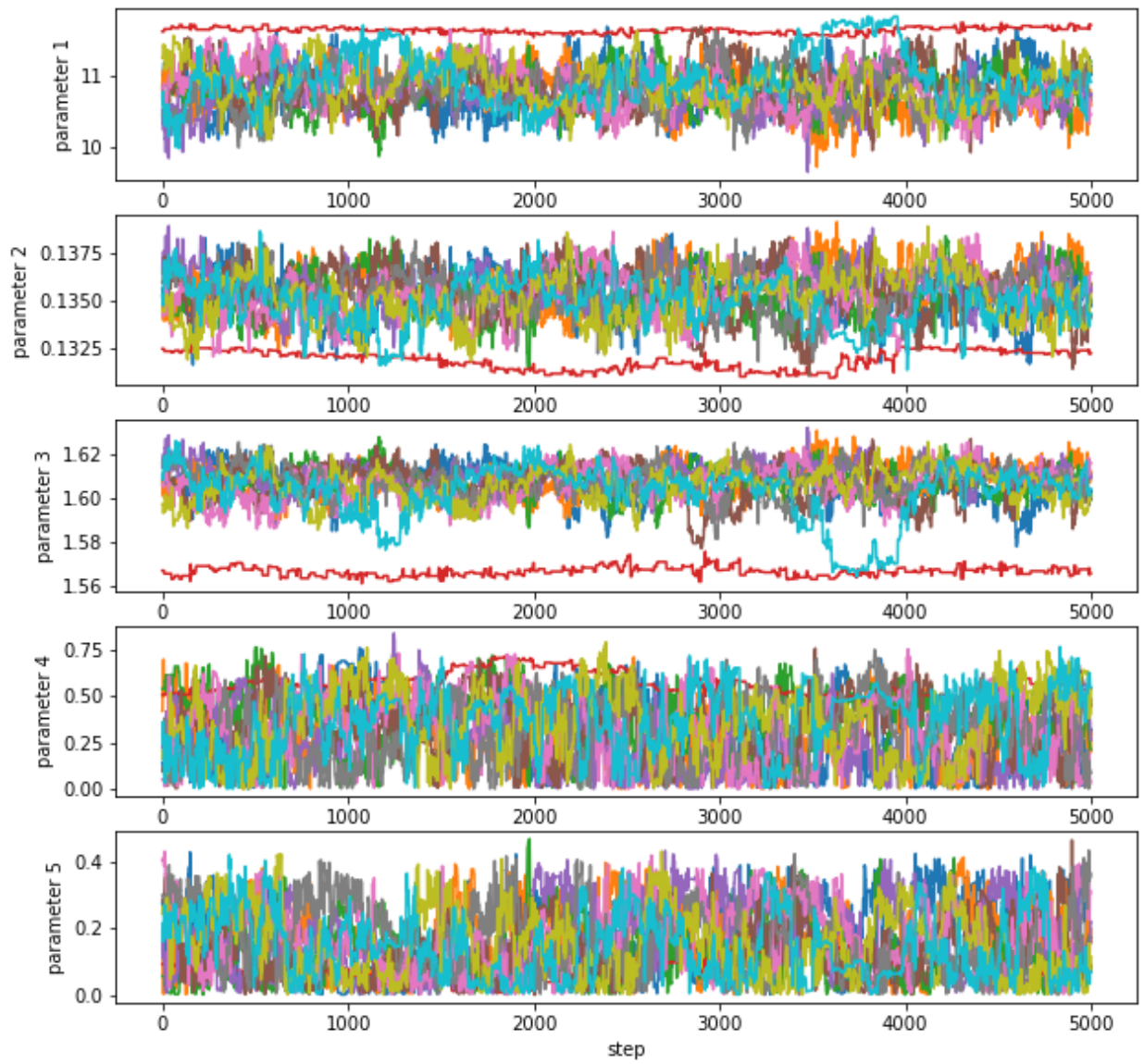
main run

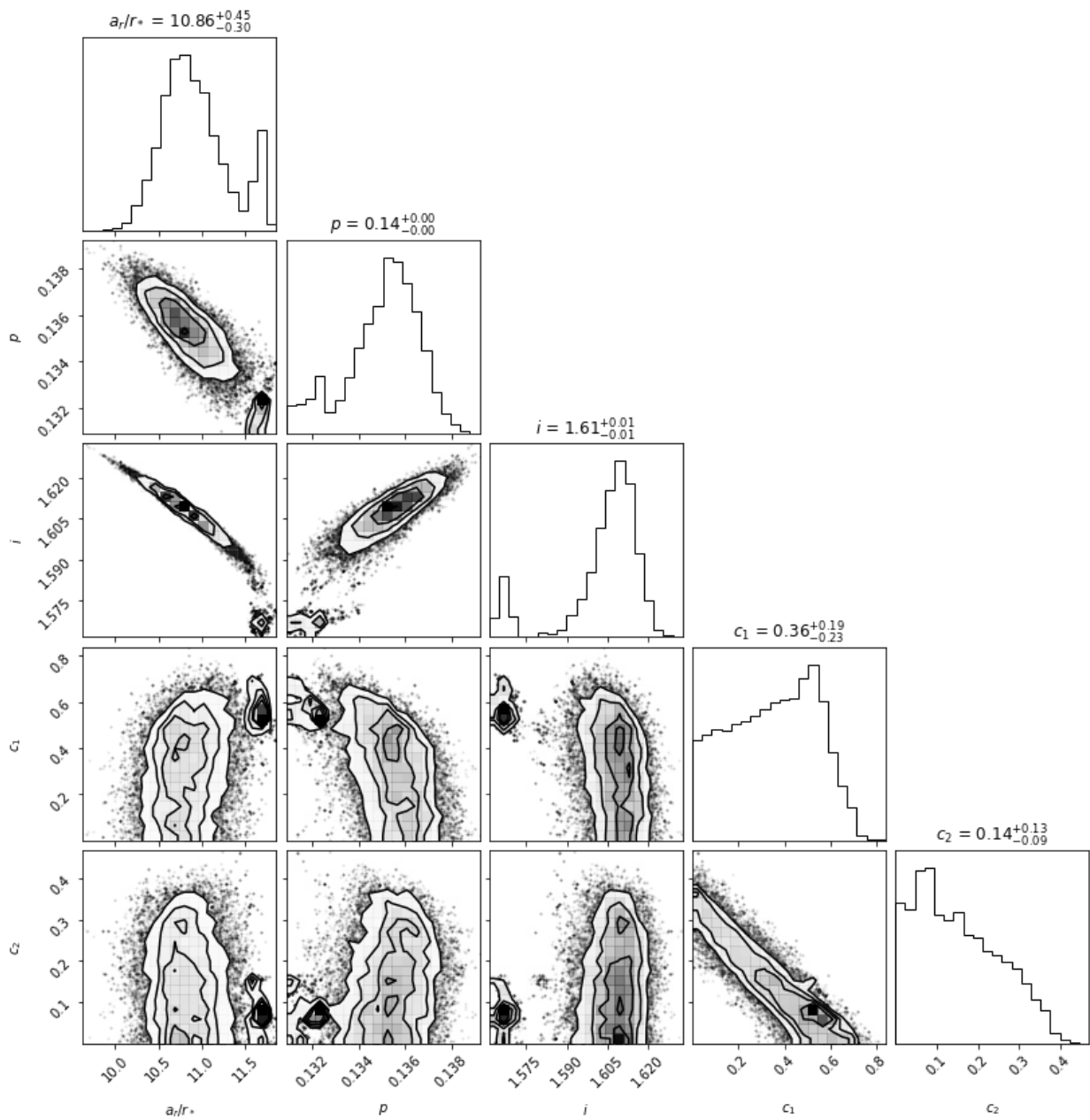
```
/var/folders/1h/wvml1fys14jd28h4cky_5hgm0000gn/T/ipykernel_7107/1939529578.py:
52: UserWarning: Matplotlib is currently using module://matplotlib_inline.back
end_inline, which is a non-GUI backend, so cannot show the figure.
```

```
f.show()
```

```
/var/folders/1h/wvml1fys14jd28h4cky_5hgm0000gn/T/ipykernel_7107/1939529578.py:
54: UserWarning: Matplotlib is currently using module://matplotlib_inline.back
end_inline, which is a non-GUI backend, so cannot show the figure.
```

```
f.show()
```



In []: